Chatbot for FAQs

CodeAlpha AI Internship — Task 2

Project type	Artificial Intelligence / Python
Deliverable	Project report with implementation source code
Prepared for	CodeAlpha AI Internship Task

Implementation note

This document includes a working starter implementation and the steps needed to run and test it. Update the sample content, test it in your own environment, and add your own screenshots or observations before submitting. Do not claim a test result that you have not personally verified.

1. Introduction

The College FAQ Chatbot answers common student questions using a small, editable collection of frequently asked questions. It uses Natural Language Processing (NLP) to compare the user's question with stored FAQ questions and returns the answer with the closest match.

The implementation uses TF-IDF to represent text as weighted numerical features and cosine similarity to measure how close the question vectors are.

2. Objectives

• Store common questions and their answers.
• Clean and vectorize question text using an NLP technique.
• Compare a user's question with stored questions.
• Return the answer for the best match.
• Provide a chat interface and a fallback for unknown questions.

3. System requirements

Python 3.9 or later. Install packages with **pip install streamlit scikit-learn**. No external API key is required.

4. NLP methodology

TF-IDF assigns importance to words based on their frequency in a question and their rarity across the FAQ collection. The vectorizer uses unigrams and bigrams. Cosine similarity compares the incoming question vector with each stored FAQ vector. The highest-scoring FAQ is selected. A threshold of 0.12 is used to avoid returning an unrelated answer when no close match is found.

5. Dataset

The sample dataset contains eight general college-service FAQs covering courses, office contact, academic calendar, leave, exam timetable, fees, bonafide certificates, and placement updates. These answers are generic placeholders. Replace them with verified information from the relevant institution before real use.

6. How to run

Save the appendix code as **faq_chatbot.py**. Install the packages and run **streamlit run faq_chatbot.py**. Use the local browser URL shown in the terminal. Ask a question similar to one in the sample FAQ list.

7. Testing checklist

Test an exact FAQ question, a paraphrased question, a question with different capitalization, and an unrelated question. Check whether the fallback appears for unrelated questions. Add verified FAQs and repeat the tests.

8. Limitations and future enhancement

This is a retrieval-based chatbot, not a generative conversational model. It can fail when a question uses vocabulary very different from the stored FAQs. Accuracy depends on the FAQ dataset and threshold. Improvements include a larger verified dataset, stemming/lemmatization, multilingual support, an admin interface, and evaluation with a labelled test set.

9. Conclusion

The project demonstrates a simple NLP-based FAQ chatbot using TF-IDF and cosine similarity, with a chat interface and a safe fallback when a suitable answer is not found.

Appendix: Complete source code

```
"""
FAQ Chatbot using TF-IDF and cosine similarity
Install: pip install streamlit scikit-learn
Run: streamlit run faq_chatbot.py
"""
import streamlit as st
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

FAQS = [
    ("What courses are offered?",
     "The college offers undergraduate programmes in engineering and related disciplines."),
    ("How can I contact the college office?",
     "Contact the college office through the official phone number or email published on the college website."),
    ("Where can I find the academic calendar?",
     "The academic calendar is normally shared through the student portal or department notice board."),
    ("How do I apply for a leave?",
     "Follow your department's leave procedure and submit the required request to the class in-charge."),
    ("How can I get my exam timetable?",
     "Check official examination notices, the student portal, or department announcements."),
    ("Where can I get my fee details?",
     "Refer to the student fee portal or contact the accounts section for the latest fee information."),
    ("How do I get a bonafide certificate?",
     "Submit a bonafide certificate request through the college's prescribed office or student service process."),
    ("Where can I find placement updates?",
     "Check placement-cell announcements and the official student communication channels.")
]

questions = [q for q, _ in FAQS]
answers = [a for _, a in FAQS]
vectorizer = TfidfVectorizer(lowercase=True, stop_words="english", ngram_range=(1, 2))
faq_matrix = vectorizer.fit_transform(questions)

def find_answer(user_question):
    user_vector = vectorizer.transform([user_question])
    scores = cosine_similarity(user_vector, faq_matrix).flatten()
    best_index = scores.argmax()
    confidence = float(scores[best_index])
    if confidence &lt; 0.12:
        return ("I could not find a close match. Please rephrase your question "
                "or contact the college office.", confidence, None)
    return answers[best_index], confidence, questions[best_index]

st.set_page_config(page_title="College FAQ Chatbot", page_icon="■")
st.title("College FAQ Chatbot")
st.caption("Ask a question about common college services.")
if "chat" not in st.session_state:
    st.session_state.chat = []
for role, message in st.session_state.chat:
    with st.chat_message(role):
        st.write(message)

prompt = st.chat_input("Type your question...")
if prompt:
    st.session_state.chat.append(("user", prompt))
    answer, score, matched_question = find_answer(prompt)
    st.session_state.chat.append(("assistant", answer))
    st.rerun()

with st.sidebar:
    st.subheader("Sample questions")
    for question in questions:
        st.write("• " + question)
    if st.button("Clear conversation"):
        st.session_state.chat = []
        st.rerun()
```